

Secure Web Application - Security Hardening Project

Task Overview

This task demonstrates a complete security assessment and hardening of a vulnerable web application. Starting from a simple User Management System with multiple security flaws, we identified, documented, and fixed vulnerabilities following industry best practices. The application is built with Node.js, Express, and SQLite.

Project Duration: 3 Weeks

Tools Used: Node.js, Express, SQLite, OWASP ZAP, Nmap, Winston, Helmet.js, JWT, Bcrypt

Quick Start

Prerequisites

- Node.js (v16 or higher)
- npm (v8 or higher)
- OpenSSL (for SSL certificates)

Installation

```
bash
```

```
# Clone the repository
```

```
git clone https://github.com/your-username/secure-web-application.git
```

```
cd secure-web-application
```

```
# Install dependencies
```

```
npm install
```

```
# Generate SSL certificates (if not present)
```

```
openssl req -x509 -newkey rsa:4096 -keyout server.key -out server.cert -days 365 -nodes -subj "/CN=localhost"
```

```
# Start the server
```

```
node server.js
```

Access the Application

- **URL:** `https://localhost:3000`
 - **Test Credentials:** Register a new user or use the default `admin` (password: `Admin@123`) if seeded.
-

Security Features Implemented

Week 1: Vulnerability Assessment

- OWASP ZAP automated scanning
- Manual XSS testing with browser DevTools
- SQL Injection testing
- Comprehensive vulnerability documentation

Week 2: Security Implementation

- **Password Security:** bcrypt hashing with salt rounds (10+)
- **Input Validation:** Validator.js for email, password strength, sanitization
- **SQL Injection Prevention:** Parameterized queries (prepared statements)
- **XSS Prevention:** xss library for HTML sanitization
- **Secure Headers:** Helmet.js (CSP, XFO, HSTS, etc.)
- **Authentication:** JWT with expiry (2 hours)
- **Session Security:** HttpOnly, Secure, SameSite cookies

Week 3: Advanced Security & Logging

- **Penetration Testing:** Nmap scanning, manual SQLi/XSS testing
 - **Logging:** Winston with dual transport (console + file)
 - **Security Monitoring:** Real-time attack detection (SQLi, XSS patterns)
 - **Error Handling:** Proper error logging without stack traces in production
 - **Health Check:** `/health` endpoint for monitoring
-

Security Checklist

Category	Status
HTTPS with SSL/TLS	Implemented
Password Hashing (bcrypt)	Implemented
SQL Injection Protection	Implemented
XSS Protection	Implemented
CSRF Protection (SameSite cookies)	Implemented
Security Headers (Helmet)	Implemented
JWT Authentication	Implemented
Input Validation	Implemented
Input Sanitization	Implemented
Security Logging	Implemented
Error Handling	Implemented
Environment Variables	Implemented
RBAC (Admin/User)	Implemented

Testing

Manual Security Tests

```
bash
# Test weak password rejection
curl -X POST https://localhost:3000/register \
  -H "Content-Type: application/json" \
  -d '{"username":"test","password":"weak"}' \
  -k

# Test SQL Injection (should fail on secure endpoint)
curl -X POST https://localhost:3000/login_secure \
  -H "Content-Type: application/json" \
  -d '{"username":"admin\" OR \"1\"=\"1","password":"anything"}' \
  -k

# Test JWT authentication
# 1. Login to get token
```

```
curl -X POST https://localhost:3000/login-jwt \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"Admin@123"}' \
-k
```

```
# 2. Access protected route with token
curl -X GET https://localhost:3000/dashboard-jwt \
-H "Authorization: Bearer YOUR_TOKEN_HERE" \
-k
```

Nmap Port Scan

```
bash
nmap -sS -sV -p- localhost
# Expected: Port 3000 open (https)
```

Check Logs

```
bash
# View security events
cat security.log | grep -i "warning|attack"

# View authentication attempts
cat security.log | grep -i "login|register"
```

Vulnerability Fix Summary

Vulnerability	Severity	Fix
SQL Injection	Critical	Parameterized queries
Stored XSS	High	xss sanitization library
Weak Password Storage	High	bcrypt hashing
Missing Security Headers	Medium	Helmet.js
Insecure Session Cookies	Medium	Secure flags (HttpOnly, Secure)
Information Disclosure	Low	Error handling, NODE_ENV
No Authentication for Admin	High	JWT + RBAC

Total Vulnerabilities Found: 7
Total Vulnerabilities Fixed: 7
Fix Rate: 100%

File Structure

text

secure-web-application/

```
|— server.js      # Main application with all security features
|— package.json   # Dependencies
|— .env           # Environment variables (not committed)
|— .gitignore     # Git ignore rules
|— database.sqlite # SQLite database
|— security.log    # Security event logs
|— error.log       # Error logs
|— README.md      # This file
|— WEEK1_Assessment.md # Vulnerability assessment report
|— WEEK2_Implementation.md # Security implementation details
|— WEEK3_Final_Report.md # Final report with testing results
└— public/        # Frontend files
    |— index.html
    |— login.html
    |— register.html
    |— dashboard.html
    |— dashboard-jwt.html
    |— admin.html
    |— style.css
    └— script.js
```

Configuration

Environment Variables (.env)

env

NODE_ENV=development

JWT_SECRET=your-super-secret-jwt-key-change-this

```
SESSION_SECRET=your-super-secret-session-key-change-this  
PORT=3000
```

SSL Certificate Generation

```
bash  
# Generate self-signed certificate for local testing  
openssl req -x509 -newkey rsa:4096 -keyout server.key -out server.cert -days 365 -nodes
```

Note: In production, use certificates from a trusted CA (Let's Encrypt, DigiCert, etc.)

Logging

Winston Configuration

- **Console Transport:** Real-time console output (development)
- **File Transport:** security.log (all logs, JSON format)
- **Error Transport:** error.log (errors only, JSON format)

Log Categories

- info: Successful operations (login, registration, data access)
 - warn: Security events (SQLi attempts, XSS attempts, failed logins)
 - error: System errors (database failures, server errors)
 - debug: Development debugging (if enabled)
-

Security Best Practices Applied

1. **Input Validation:** All user inputs validated on server-side
2. **Output Encoding:** HTML escaping for user-generated content
3. **Parameterized Queries:** SQL injection protection
4. **Password Hashing:** bcrypt with 10+ salt rounds
5. **JWT Authentication:** Stateless, signed tokens with expiration
6. **Security Headers:** Helmet.js with CSP
7. **HTTPS:** SSL/TLS encryption for all traffic
8. **Session Security:** Secure, HttpOnly, SameSite cookies
9. **Error Handling:** No stack traces in production

10. **Logging:** Comprehensive security event logging
 11. **RBAC:** Role-based access control (Admin/User)
 12. **Environment Variables:** No hardcoded secrets
-

Technologies Used

Technology	Purpose
Node.js	Runtime environment
Express.js	Web framework
SQLite	Database (development)
bcrypt	Password hashing
jsonwebtoken	JWT authentication
helmet	Security headers
validator	Input validation
xss	XSS sanitization
winston	Logging
sqlite3	Database driver
crypto	Built-in crypto (Node.js)

Future Improvements

- Implement rate limiting (express-rate-limit)
- Add 2FA/MFA support
- Implement password reset functionality
- Add CSRF tokens
- Implement CORS properly
- Add API documentation (Swagger)
- Set up CI/CD with security scanning
- Containerize with Docker
- Add automated testing (Jest/Mocha)
- Implement OAuth2/Social login